

NewtonImplicitSolver

August 22, 2026

This file is part of CasADi.

CasADi -- A symbolic framework for dynamic optimization.
Copyright (C) 2010–2023 Joel Andersson, Joris Gillis, Moritz Diehl,
KU Leuven. All rights reserved.
Copyright (C) 2011–2014 Greg Horn

CasADi is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 3 of the License, or (at your option) any later version.

CasADi is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public
License along with CasADi; if not, write to the Free Software
Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA

1 NewtonImplicitSolver

```
[1]: import casadi as ca
      from numpy import *
      from pylab import *
```

We will investigate the working of rootfinder with the help of the parametrically excited Duffing equation. $\ddot{u} + \dot{u} - \epsilon(2\mu\dot{u} + \alpha u^3 + 2ku \cos(\Omega t))$ with $\Omega = 2 + \epsilon\sigma$. \

The first order solution is $u(t) = a \cos(\frac{1}{2}\Omega t - \frac{1}{2}\gamma)$ with the modulation equations: $\frac{da}{d\epsilon t} = -[\mu a + \frac{1}{2}ka \sin \gamma]$ \ $a \frac{d\gamma}{d\epsilon t} = -[-\sigma a + \frac{3}{4}\alpha a^3 + ka \cos \gamma]$ \

We seek the stationary solution to these modulation equations.

Parameters

```
[2]: eps    = ca.SX.sym("eps")
      mu     = ca.SX.sym("mu")
```

```
alpha = ca.SX.sym("alpha")
k      = ca.SX.sym("k")
sigma = ca.SX.sym("sigma")
params = [eps,mu,alpha,k,sigma]
```

Variables

```
[3]: a      = ca.SX.sym("a")
      gamma = ca.SX.sym("gamma")
```

Equations

```
[4]: res0 = mu*a+1.0/2*k*a*sin(gamma)
      res1 = -sigma * a + 3.0/4*alpha*a**3+k*a*cos(gamma)
```

```
/casadi_python/casadi/casadi.py:913: FutureWarning:
casadi: a numpy function was called on a casadi value (issue #2959).
This used legacy casadi 3.7.2 behaviour, where the result type follows
the input:
```

- a numeric input (DM) -> a plain numpy array
- a symbolic input (SX / MX) -> a casadi value, or an error if the function has no casadi equivalent

For a shape-correct, type-preserving result, opt in to the casadi-aware numpy support:

```
try:
    ca.GlobalOptions.setNumpyMode(1)
except AttributeError:
    pass
```

Or use `setNumpyMode(-1)` to keep legacy behaviour silently.
Shown once; run python with `-W` always to see every occurrence.

```

w.warn( _NUMPY_LEGACY_NOTICE, FutureWarning)

```

Numerical values

```
[5]: sigma_ = 0.1
      alpha_ = 0.1
      k_ = 0.2
      params_ = [0.1,0.1,alpha_,k_,sigma_]
```

We create a `NewtonImplicitSolver` instance

```
[6]: f=ca.Function("f", [ca.vertcat(a,gamma),ca.vertcat(*params)],[ca.  
    ↵vertcat(res0,res1)])  
opts = {}  
opts["abstol"] = 1e-14
```

```
opts["linear_solver"] = "csparse"  
s=ca.rootfinder("s", "newton", f, opts)
```

Initialize $[a, \gamma]$ with a guess and solve

```
[7]: x_ = s([1,-1], params_)
```

```
[8]: print("Solution = ", x_)
```

Solution = [1.1547, -1.5708]

Compare with the analytic solution:

```
[9]: x = [sqrt(4.0/3*sigma_/alpha_), -0.5*pi]  
print("Reference solution = ", x)
```

Reference solution = [1.1547005383792515, -1.5707963267948966]

We show that the residual is indeed (close to) zero

```
[10]: residual = f(x_, params_)  
print("residual = ", residual)
```

residual = [1.29063e-15, 2.62584e-15]

```
[11]: for i in range(1):  
      assert(abs(x_[i]-x[i])<1e-6)
```

Solver statistics

```
[12]: print(s.stats())
```

```
{'iter_count': 29, 'n_call_g': 0, 'n_call_jac_g_x': 0, 'return_status':  
'success', 'success': True, 't_proc_g': 0.0, 't_proc_jac_g_x': 0.0, 't_wall_g':  
0.0, 't_wall_jac_g_x': 0.0, 'unified_return_status': 'SOLVER_RET_UNKNOWN'}
```